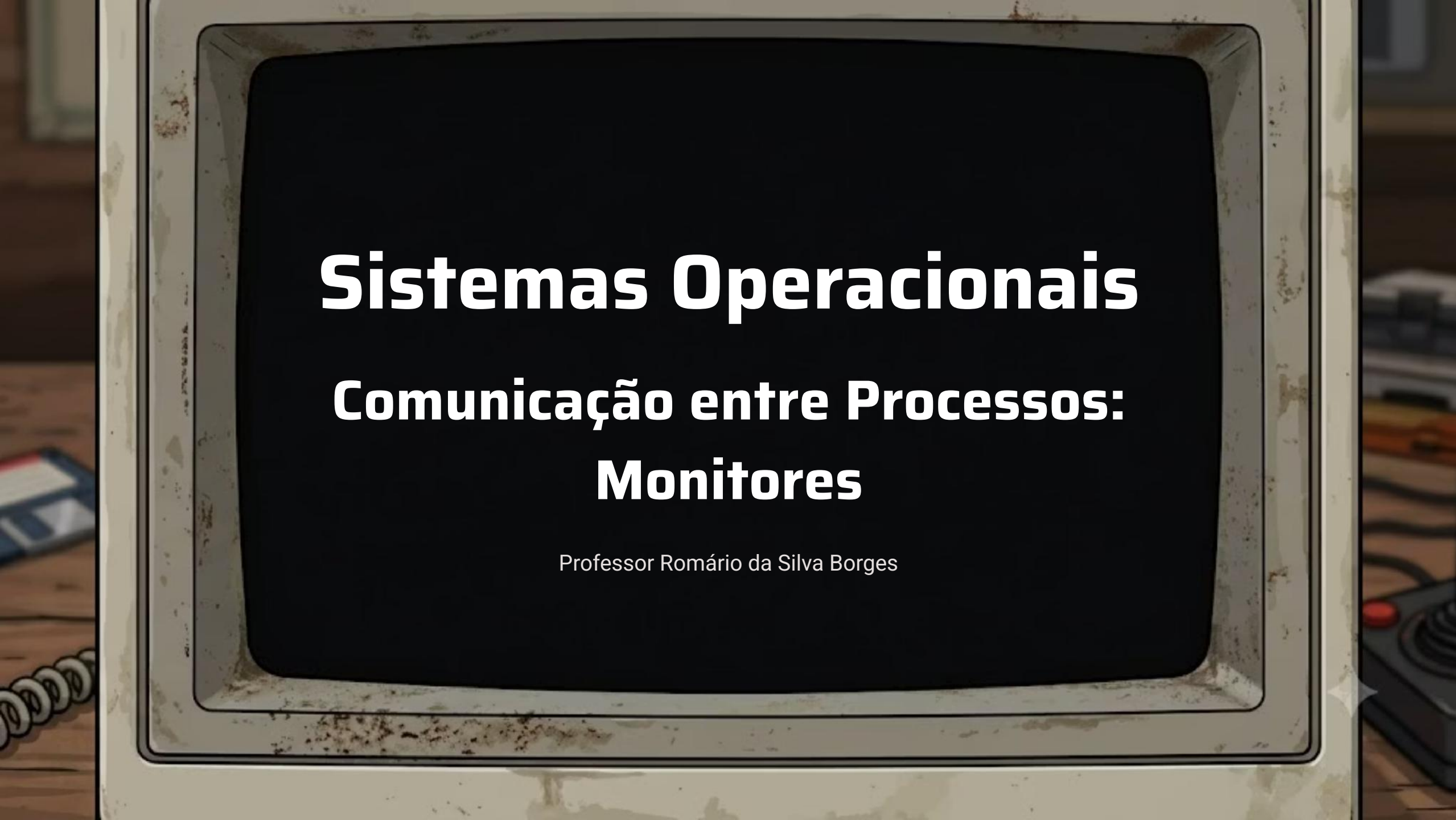




Sistemas Operacionais

Comunicação entre Processos:

Monitores

Professor Romário da Silva Borges

Por que monitores?

Semáforos são eficazes.

Mas exigem uso **correto** de `down/up`.

Erros sutis

Pequenos erros podem quebrar toda a sincronização.

Difícil de testar

Erros aparecem apenas em certas sequências de execução, dificultando testes e depuração.

Erro comum com semáforos

✓ Sequência correta

```
down(mutex)  
    região crítica  
up(mutex)
```

✗ Sequência incorreta

```
up(mutex)  
    região crítica  
down(mutex)
```

Problema

Com a sequência incorreta, **todos os processos** podem entrar na região crítica simultaneamente.

A questão central

Semáforos resolvem o problema.

Mas a pergunta é:

Será simples criar soluções corretas com semáforos em todos os casos?

Resposta

Nem sempre.

Origem dos monitores



C. A. R. Hoare

Proposto em **1974** como unidade de sincronização de alto nível.



Per Brinch Hansen

Formalizado em **1975** com foco em facilitar soluções corretas.

Objetivo

Facilitar a criação de soluções corretas para sincronização de processos.

O que é um monitor?

Monitor é um módulo que agrupa funções, variáveis e estruturas de dados.



Funções

Operações sobre os dados compartilhados.



Variáveis

Estado interno protegido do monitor.



Estruturas de dados

Dados compartilhados encapsulados.

Monitor como TAD

Dados protegidos

Os dados internos ficam protegidos e não são acessíveis diretamente.

Acesso por métodos

O acesso ocorre apenas pelos procedimentos definidos no monitor.

Encapsulamento

Detalhes internos ficam ocultos — como em um Tipo Abstrato de Dados.

Um monitor pode ser visto como um **Tipo Abstrato de Dados (TAD)**.

Encapsulamento

✓ Processos podem:

- Chamar procedimentos do monitor
- Usar as operações fornecidas

✗ Processos NÃO podem:

- Acessar variáveis internas
- Acessar estruturas internas
- Manipular dados compartilhados diretamente

Benefício

O encapsulamento **reduz o risco de uso incorreto** dos dados compartilhados.

Exemplo conceitual de monitor

```
monitor NomeDoMonitor {  
  
    variáveis privadas;  
  
    método1() {  
        // operação sobre dados internos  
    }  
  
    método2() {  
        // operação sobre dados internos  
    }  
}
```

Ideia

As operações sobre dados compartilhados ficam **concentradas e protegidas** dentro do monitor.

Exclusão mútua no monitor

Apenas um processo ou thread pode estar ativo dentro do monitor por vez.

Se já há um processo ativo

O novo processo será **bloqueado** e aguardará o monitor ficar disponível.

Quando o processo sai

O monitor fica **disponível** e outro processo pode entrar.

Quem controla a exclusão mútua?

Nos monitores, a exclusão mútua é controlada pela linguagem, compilador ou interpretador.

Vantagem

O programador **não precisa inserir manualmente** down/up em cada entrada da região crítica.

Papel do compilador

O compilador sabe que o monitor é **especial**. Ao chamar um procedimento do monitor:

1

Verifica

Há outro processo ativo dentro do monitor?

2

Livre?

Libera a entrada para o processo.

3

Ocupado?

Bloqueia o processo até ficar disponível.

Por que isso reduz erros?

Com semáforos:

- O programador controla `down/up`
- Risco de esquecer ou inverter
- Risco de posicionar incorretamente

Com monitores:

- A entrada é controlada **automaticamente**
- Menor chance de erro
- Programador foca na lógica

Resultado

Menor chance de esquecer, inverter ou posicionar incorretamente as operações de sincronização.

Monitor e semáforo binário

Internamente

Uma implementação simples de monitor pode usar um **semáforo binário** internamente.

A diferença está na abstração

Semáforo: controle explícito pelo programador.

Monitor: controle encapsulado pela linguagem.

Conceito-chave

Mesma base, nível de abstração diferente.

Linguagens sem suporte direto

Em linguagens como **C** e **Pascal**, os compiladores **não implementam monitores** diretamente.

01

Usar bibliotecas

Bibliotecas de sincronização que emulam o comportamento de monitores.

02

Usar semáforos

Implementar manualmente o controle equivalente ao de um monitor.

03

Simular no espaço de usuário

Criar a lógica de controle de acesso dentro da própria aplicação.

Monitores em Java

Em Java, todo objeto possui um monitor associado.

O monitor como um "passe"

Apenas **uma thread** pode possuir esse passe em determinado instante.

Associado ao objeto

Cada objeto Java carrega seu próprio monitor — pronto para ser usado.

Como pedir o monitor em Java?

synchronized

Em métodos

Protege o método inteiro — a thread precisa adquirir o monitor do objeto para executar.

Em blocos de código

Protege apenas um trecho específico do código dentro de um método.

synchronized em métodos

1

Tenta adquirir

A thread tenta adquirir o monitor do objeto.

2

Executa ou bloqueia

Se conseguir, executa. Se não, fica bloqueada.

3

Libera

Ao sair do método, o monitor é liberado automaticamente.

Exemplo: método synchronized

```
public synchronized void alterarDados() {  
    // região protegida  
}
```

Garantia

Apenas **uma thread por vez** executa esse método no mesmo objeto.

synchronized em blocos

Também é possível sincronizar apenas um **trecho do código**:

```
synchronized(objeto) {  
    // região protegida  
}
```

Vantagem

Protege apenas a parte necessária — **maior granularidade de controle**.

Vantagem do bloco synchronized



Menor tempo bloqueado

O monitor é segurado por um período menor de tempo.



Mais flexibilidade

É possível definir exatamente qual região será protegida.



Ganho de desempenho

Permite maior paralelismo nas partes não protegidas do código.

Cuidado com blocos synchronized

A flexibilidade também pode gerar **desorganização**.

Riscos

- Código menos coeso
- Uso do monitor de **outro objeto**
- Maior chance de erro de projeto
- Dificuldade de manutenção

Monitores × Semáforos

Semáforos

- Mecanismo mais **básico**
- Dependem de **down/up** corretos
- São flexíveis
- Podem gerar erros difíceis de detectar

Monitores

- Mecanismo de **mais alto nível**
- Encapsulam dados e operações
- Exclusão mútua **automática**
- Reduzem erros comuns

Quando monitores fazem sentido?



Há dados compartilhados

Quando múltiplos processos/threads acessam os mesmos dados.



Encapsulamento é importante

Quando o acesso deve ser centralizado e protegido.



Reduzir erros de sincronização

Quando se quer minimizar riscos de uso incorreto.



Código mais organizado

Quando a linguagem oferece suporte e o código precisa de clareza.

Limitação dos monitores

Monitores ajudam na **exclusão mútua**.
Mas ainda exigem cuidado.

Condições de espera

O projeto das condições de espera ainda é responsabilidade do programador.

Ordem das chamadas

A sequência de chamadas pode gerar bloqueios indesejados.

Complexidade concorrente

Interação entre objetos em aplicações concorrentes complexas ainda exige atenção.

Referências bibliográficas

TANENBAUM, Andrew S.; BOS, Herbert.

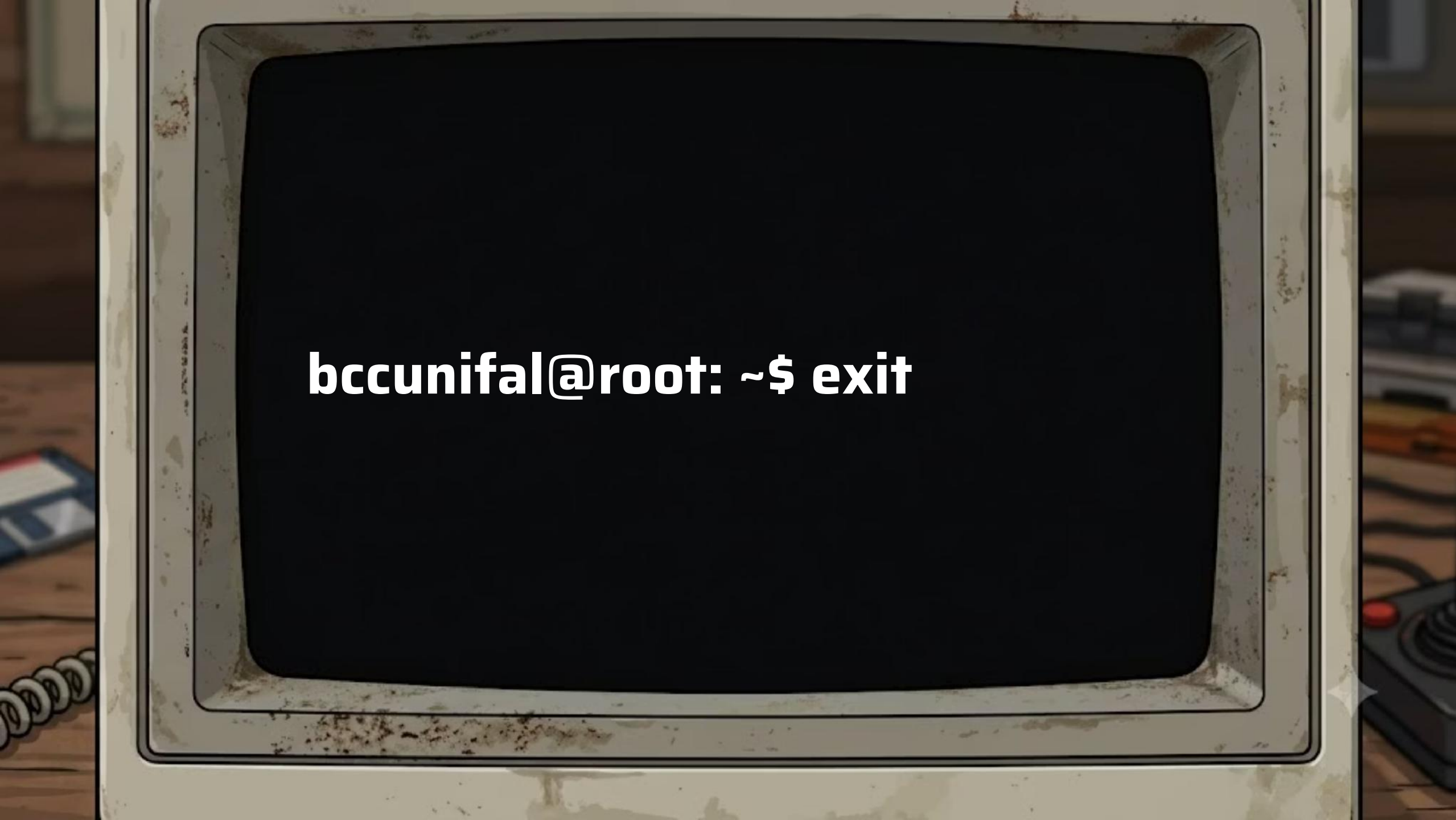
Sistemas Operacionais Modernos. 4. ed. São Paulo: Pearson, 2016.

SILBERSCHATZ, Abraham; GALVIN, Peter B.; GAGNE, Greg.

Fundamentos de Sistemas Operacionais. 9. ed. Rio de Janeiro: LTC, 2015.

MAZIERO, Carlos A.

Sistemas Operacionais: Conceitos e Mecanismos. Curitiba: UFPR, 2019.

A terminal window is displayed on a computer monitor. The monitor has a light-colored, slightly textured bezel. The terminal window itself is a dark rectangle with a thin white border. Inside the terminal, the text 'bccunifal@root: ~\$ exit' is written in a white, monospaced font. The background of the entire image shows a desk with a keyboard and some papers, though they are out of focus.

bccunifal@root: ~\$ exit